

Guia para Desenvolvedores após a Migração (Azure DevOps Git)

Como clonar, trabalhar, commitar e lidar com submodules

1) O que muda depois da migração

Após a migração, o SVN deixa de ser a fonte oficial. O trabalho diário passa a ser feito diretamente no Azure DevOps Git: clone, branch, commits, push e pull requests. Dependências que antes eram svn:externals passam a ser submodules Git.

2) Primeiro clone (monorepo)

- Use Git for Windows (Git Bash).
- Se você usa pastas profundas, habilite longpaths no Windows (Policy) e no Git.

```
# clone do monorepo
git clone https://dev.azure.com/<ORG>/<PROJECT>/_git/<MONOREPO>
cd <MONOREPO>

# inicializa e baixa os submodules (primeira vez)
git submodule update --init --recursive
```

Dica: atualizar submodules para o SHA fixado no monorepo

```
git submodule update --init --recursive
```

3) Fluxo normal de trabalho (sem mexer em submodules)

```
# criar branch
git checkout -b feature/minha_mudanca

# editar arquivos...
git status
git add .
git commit -m "Descrição curta e objetiva"
git push -u origin feature/minha_mudanca
```

Depois disso, abra um Pull Request no Azure DevOps e siga o fluxo de revisão da equipe.

4) Quando você precisa alterar um submodule (ex.: Comum)

Submodules são repositórios separados. Se você alterar arquivos dentro de um submodule, você precisa fazer commit/push dentro dele, e depois atualizar o monorepo para apontar para o novo SHA.

Passo a passo

1. Entre na pasta do submodule.
2. Crie branch, commit e push dentro do submodule (repo próprio).
3. Volte ao monorepo. Ele vai mostrar que o submodule mudou (novo SHA).
4. Faça commit no monorepo atualizando o ponteiro do submodule e dê push.

```
# 1) dentro do submodule
cd <caminho/do/submodule>
git checkout -b fix/comum
# editar...
git add .
git commit -m "Ajuste no Comum"
git push -u origin fix/comum

# 2) volta ao monorepo e 'bump' do SHA
cd <raiz/monorepo>
git status # vai mostrar o submodule como modificado
git add <caminho/do/submodule>
git commit -m "Bump submodule <nome> para novo SHA"
git push
```

5) Comandos Git úteis (cola rápida)

- Verificar status: `git status`
- Ver branches: `git branch -a`
- Atualizar branch local: `git pull --rebase`
- Ver log curto: `git log --oneline --decorate -n 20`
- Descartar mudanças locais: `git restore .`
- Resolver conflito: edite arquivos + `git add` + `git rebase --continue` (ou `merge`).
- Ver submodules: `git submodule status`

6) Boas práticas (para não sofrer)

- Evite editar submodule por engano. Sempre confira onde você está (pwd) antes do commit.
- Submodule é SHA fixo: seu projeto só atualiza o Comum quando alguém fizer o bump no monorepo.
- Não rode git submodule update --remote a menos que a equipe use isso como padrão.
- Padronize mensagens de commit e use PRs sempre que possível.

7) Problemas comuns no dia a dia

Clonou e faltou conteúdo do submodule

```
git submodule update --init --recursive
```

Submodule aponta para SHA que você não tem localmente

- Rode git fetch dentro do submodule.
- Se persistir, verifique se você tem permissão no repo do submodule no Azure.

Erro de caminho longo (Windows)

```
git config --global core.longpaths true
```

Addendum — Clone parcial + Workspace VB6 (Junctions)

Este addendum adiciona um modo de **baixar só 1 projeto** (sem trazer o monorepo inteiro) e o passo oficial de **junctions** para abrir/compilar VB6 sem editar milhares de .VBP.

1) Clone parcial (somente 1 projeto/área)

Use **sparse-checkout** para trazer apenas o caminho necessário. Se combinado com **partial clone** (`--filter=blob:none`), reduz o download de arquivos.

Exemplo recomendado (sparse + partial)

```
git clone --filter=blob:none --no-checkout <URL_DO_MONOREPO>
"C:\fontes\monorepo_culturas"
cd "C:\fontes\monorepo_culturas"
git sparse-checkout init --cone
git sparse-checkout set "Culturas/Agricola/GATec_GRP_Dev"
git checkout main
git submodule update --init --recursive
```

- Troque o caminho por sua área/projeto (ex.: **Culturas/Pecuaria/...**).
- Se precisar de múltiplas áreas, use **git sparse-checkout set** com vários caminhos.

2) Workspace VB6 (Junctions + Stubs)

Quando o VB6 acusa “Path not found” ao abrir um .VBP, rode o setup para criar junctions que simulam a topologia esperada (externals do SVN) sem mexer no repo.

Dry-run (simulação)

```
.\setup_workspace_junctions.ps1 `
-VbpPath
"C:\fontes\monorepo_culturas\Culturas\Agricola\GATec_GRP_Dev\GATec_GRP\GATec_GRP.vbp"
`
-SearchDepth 25 `
-DryRun
```

Execução real (recomendado) + stubs para comuns fora de escopo

```
.\setup_workspace_junctions.ps1 `
-VbpPath
"C:\fontes\monorepo_culturas\Culturas\Agricola\GATec_GRP_Dev\GATec_GRP\GATec_GRP.vbp"
`
-SearchDepth 25 `
-CreateStubsForMissingCommons
```

3) Rodar para vários projetos (lote)

Se você quiser preparar uma área inteira (ex.: Agricola) para abrir diversos projetos VB6 sem repetir manualmente:

```
.\setup_workspace_junctions_batch.ps1 `
-RootDir "C:\fontes\monorepo_culturas\Culturas\Agricola" `
-SetupScriptPath "C:\fontes\monorepo_culturas\setup_workspace_junctions.ps1" `
-SearchDepth 25 `
```

-CreateStubsForMissingCommons

Troubleshooting rápido

- **Access is denied** → abrir PowerShell como Administrador.
- **Não resolvido** no log → comum pode estar fora do seu escopo; use **-CreateStubsForMissingCommons**.
- Se o dev está usando clone parcial, confirme que o caminho da área/projeto foi incluído no sparse-checkout.